

Assignment 5

1. To find the fastest plan for the given query, first, I will reduce I/O accesses by pushing selections down to reduce tuples early. Meaning that we will push $R.E \leq 200$ on R and $W.N \leq 100$ on W to reduce the size of those relations early. Then I will try to avoid cartesian products by observing that the six relations naturally split into two groups based on the join conditions. Group 1 being the {R, S, T} which is formed by $R.D = S.D$, $R.D = T.D$, and $R.B = T.B$; and Group 2 which is the {U, V, W} which is formed by $U.J = W.J$ and $U.J \leq V.J$. Since the cartesian product between them is inevitable because there is no direct join between them, we will try and delay it until we have no other options. This will be when the groups have been fully joined and reduced to their smallest possible size. We will then choose the best guess join order by joining within each group first before combining them. In Group 1, we join the filtered R with S, then join that result with T. In Group 2, we join the filtered W with U, then join that result with V. Finally, once both groups are fully reduced, we perform the unavoidable Cartesian product between them to produce the final result. This plan ensures the fastest execution because at every step we are minimizing the size of each subsets result. By pushing selections down first, R and W are already smaller before any joins happen. By joining each group using equi-join conditions before combining the groups, we avoid producing large intermediate results early. And by delaying the Cartesian product until the very end, we ensure that when it does happen, both sides are as small as they can possibly be. This means less data is read, written, and processed at each stage, resulting in fewer I/O accesses overall.

2. Using $T(R \bowtie S) = \frac{T(R) \cdot T(S)}{\max(V(R, A), V(S, A))}$ (Where the $\bowtie$ is a join)

$$\text{We get } T(R \bowtie S) = \frac{100,000 \cdot 10,000}{\max(100, 1000)} = 1,000,000 \text{ tuples}$$

3. Three other SQL operators that a relational query optimizer can push selection and projection below the operators in a query plan without changing the result of the query are DISTINCT, GROUP BY, and ORDER BY. Selection and projections can be pushed below DISTINCT because filtering or removing columns before eliminating duplicates produces the same final result. Similarly, selections can be pushed below GROUP BY because filtering rows before grouping produces the same groups as filtering after, since we are simply reducing the input before the grouping happens. Finally, selections and projections can be pushed below ORDER BY because filtering or removing columns before sorting does not affect the final

ordering of the result, meaning the output remains the same regardless of when the selection or projection is applied.

4. The System-R algorithm works by first considering all subsets of size i from n relations at each step i . Which then takes each subset to try all the different possible ways to split the subset into a smaller subset and only one single relation. Therefore, at step i we will have $C(n, i)$ subsets to observe, meaning we are picking i relations out of n total to consider at that step. Since all of the work summed for all the subsets is 2^n and we will try at most n splits the overall time complexity is $O(n \cdot 2^n)$